DFSha

Sistema de archivos distribuido con alta disponibilidad

Informe técnico

Versión del documento	Borrador 0.1 (documento vivo; se actualiza al integrar los componentes)
Fecha	18 de octubre de 2026
Repositorio	github.com/Valenciavm/DFS
Estado del código revisado	Commit 0bcb216 (rama principal al momento de la revisión)
Opción del proyecto	Opción 1 — Cliente/Servidor (arquitectura Maestro–Trabajadores, estilo HDFS)
Equipo	[Nombres de los integrantes]
Curso / Institución	[Curso] · [Universidad]

Cómo leer este borrador. Cada capacidad del sistema está marcada como **Implementado** (existe en el código y tiene pruebas), **Parcial**, **Diseñado** (está especificado en los documentos de arquitectura pero no escrito) o **Pendiente**. Las cifras de la sección 7 se midieron sobre el código real. La sección 8 resume qué falta para cerrar el proyecto y qué diferencias detectamos entre el contrato y el código.

Contenido

1. Objetivo y marco teórico 3

2. Descripción del servicio y problema abordado 4

3. Arquitectura del sistema y diagramas 5

4. Especificación de protocolos y APIs 9

5. Algoritmos de particionamiento y distribución 12

6. Entorno de ejecución (nativo y Docker) 14

7. Pruebas y análisis de resultados 15

8. Estado de implementación y trabajo pendiente 18

Referencias 19

1. Objetivo y marco teórico

1.1 Objetivo

Diseñar, implementar y evaluar un **sistema de archivos distribuido basado en bloques** que permita almacenar y recuperar archivos grandes sobre varias máquinas, de forma que la caída de cualquiera de ellas no provoque pérdida de datos ni interrupción del servicio, con acceso controlado y datos cifrados. El sistema debe ofrecer al usuario una vista jerárquica tipo Linux (`ls`, `cd`, `mkdir`, `put`, `get`…) y ocultar dónde está físicamente cada bloque.

Objetivos específicos:

- Particionar cada archivo en bloques y distribuirlos entre nodos (los archivos se reparten entre nodos *y* cada archivo se reparte entre varios nodos).
- Replicar cada bloque (factor 3 por defecto) y reparar automáticamente las réplicas perdidas.
- Garantizar consistencia de metadatos fuerte y visibilidad atómica de los archivos (modelo WORM).
- Proteger comunicaciones, almacenamiento y acceso (TLS/mTLS, AES-256-GCM, JWT, API keys, 2FA, ACL).
- Evaluar escalabilidad, disponibilidad y desempeño con un plan de pruebas reproducible.

1.2 Marco teórico breve

Sistemas de archivos distribuidos. Un sistema de archivos distribuido presenta como un único espacio de nombres un almacenamiento que en realidad vive en muchas máquinas. Las propiedades que se persiguen son transparencia (el usuario no ve la ubicación), escalabilidad (agregar nodos aumenta capacidad y caudal), tolerancia a fallos (la falla de componentes es la norma y no la excepción) y seguridad.

Arquitectura GFS/HDFS. El Google File System (Ghemawat et al., 2003) y su descendiente abierto HDFS (Shvachko et al., 2010) separan el *plano de control* del *plano de datos*: un nodo maestro (NameNode) guarda el espacio de nombres y el mapa archivo → bloques → réplicas, mientras que muchos nodos de datos (DataNodes) almacenan los bloques. Los clientes consultan al maestro solo por metadatos y mueven los bytes directamente con los DataNodes, por lo que el maestro no es un cuello de botella de ancho de banda. DFSha adopta este modelo.

Bloques frente a objetos. Un almacenamiento por bloques parte los archivos en fragmentos de tamaño fijo (64 MiB por defecto, igual que HDFS), lo que permite paralelizar transferencias, repartir carga y reparar a granularidad fina. El almacenamiento de objetos (estilo S3) trata el archivo completo como una unidad y no expone esta estructura al cliente.

Replicación y consistencia. Cada bloque se copia en varios nodos (replicación en cadena o *pipeline*). Con datos inmutables (WORM, *write-once-read-many*) desaparece el problema de conflictos de actualización: una réplica sirve igual que cualquier otra. Los metadatos, en cambio, requieren consistencia fuerte; DFSha la obtiene con un único líder que escribe en una base transaccional. El diseño separa *estado duro* (debe sobrevivir a todo: árbol, permisos, bloques de cada archivo) de *estado blando* (quién está vivo y dónde está cada réplica), que se reconstruye con latidos e informes de bloques.

Alta disponibilidad. Se usa elección de líder por *lease* (arrendamiento con vencimiento) sobre una base de datos compartida y un número de *término* (*term*) que funciona como *fencing*: los DataNodes ignoran órdenes de términos anteriores, lo que neutraliza a un líder obsoleto («zombi»). La detección de fallos se hace con latidos periódicos.

Seguridad. Cifrado en tránsito (TLS; mTLS entre nodos), cifrado en reposo (AES-256-GCM, que además autentica los datos), autenticación de usuarios con contraseña (Argon2id), JWT, claves de API y TOTP (RFC 6238), y autorización mediante propietario + ACL por usuario o grupo y *tokens de bloque* firmados

con HMAC que el DataNode valida localmente.

2. Descripción del servicio y problema abordado

2.1 Problema

Un solo servidor de archivos es un punto único de falla y un límite de capacidad. Cuando el volumen de datos supera lo que cabe (o lo que se puede servir) en una máquina, y cuando se exige que el servicio no se interrumpa ni se pierdan archivos ante la caída de nodos, hace falta distribuir y replicar el almacenamiento, manteniendo además control de acceso y confidencialidad.

2.2 Servicio ofrecido

DFSha es un **sistema de archivos distribuido por bloques** al estilo HDFS:

- Los archivos se dividen en **bloques** de tamaño fijo, repartidos entre distintos **DataNodes**; cada bloque se **replica** (factor 3).
- Un **NameNode** (nodo de control) mantiene todo el metadato: espacio de nombres, permisos, bloques de cada archivo y ubicación de las réplicas.
- Un **Cliente** (CLI + biblioteca Python) ofrece la vista jerárquica y oculta la ubicación de los datos.
- Los archivos son **WORM**: un archivo confirmado es inmutable; modificarlo equivale a subir una nueva versión (`overwrite`).

2.3 Mapa de requisitos

Requisito	Cómo se cubre	Estado
RF1 Gestión del sistema de archivos (`ls, cd, mkdir, rmdir, rm…`)	Endpoints `/v1/fs/*` en el NameNode; `cd` es estado del lado cliente validado con `stat`	**Parcial**
RF2 Transferencia (`put/get`)	Plan de subida del NameNode + `WriteBlock`/`ReadBlock` gRPC en los DataNodes. Falta la biblioteca cliente que los une	**Parcial**
RF3 Acceso (`open/close/read/lock`)	`ReadBlock` con rangos (`offset/length`) y locks `shared/exclusive` con expiración en el NameNode	**Parcial**
Archivos y cada archivo repartidos entre nodos	Colocación por bloque con muestreo ponderado (sección 5)	**Implementado**
Alta disponibilidad / replicación / consistencia	Pipeline, quórum en `commit`, re-replicación (implementados); líder por lease y BD replicada (pendientes)	**Parcial**
Seguridad (tránsito, reposo, acceso, entre nodos)	Tokens de bloque, JWT, API keys, ACL (implementados); TLS/mTLS, AES-GCM y TOTP (pendientes)	**Parcial**
Transparencia	El cliente nunca elige nodos; el NameNode decide	**Diseñado**
Cada nodo expone una API	NameNode: REST + OpenAPI (`/docs`). DataNode: gRPC `BlockService`	**Implementado**

Opción asignada: **Opción 1 — Cliente/Servidor**. El servidor se implementa por composición y distribución (NameNodes + DataNodes + base de datos de metadatos) dentro de una red privada, exponiendo APIs abiertas al cliente: REST para metadatos y gRPC para bloques.

3. Arquitectura del sistema y diagramas

3.1 Visión general

DFSha es una arquitectura **Maestro–Trabajadores**. El **plano de control** (NameNode, con su base de datos de metadatos) decide *dónde* va cada bloque; el **plano de datos** (DataNodes) almacena y transmite los bytes. El NameNode atiende únicamente REST; los DataNodes atienden únicamente gRPC para bloques y son quienes llaman al NameNode (registro, latidos, informes), como en HDFS, de modo que el NameNode nunca necesita alcanzar a un DataNode.

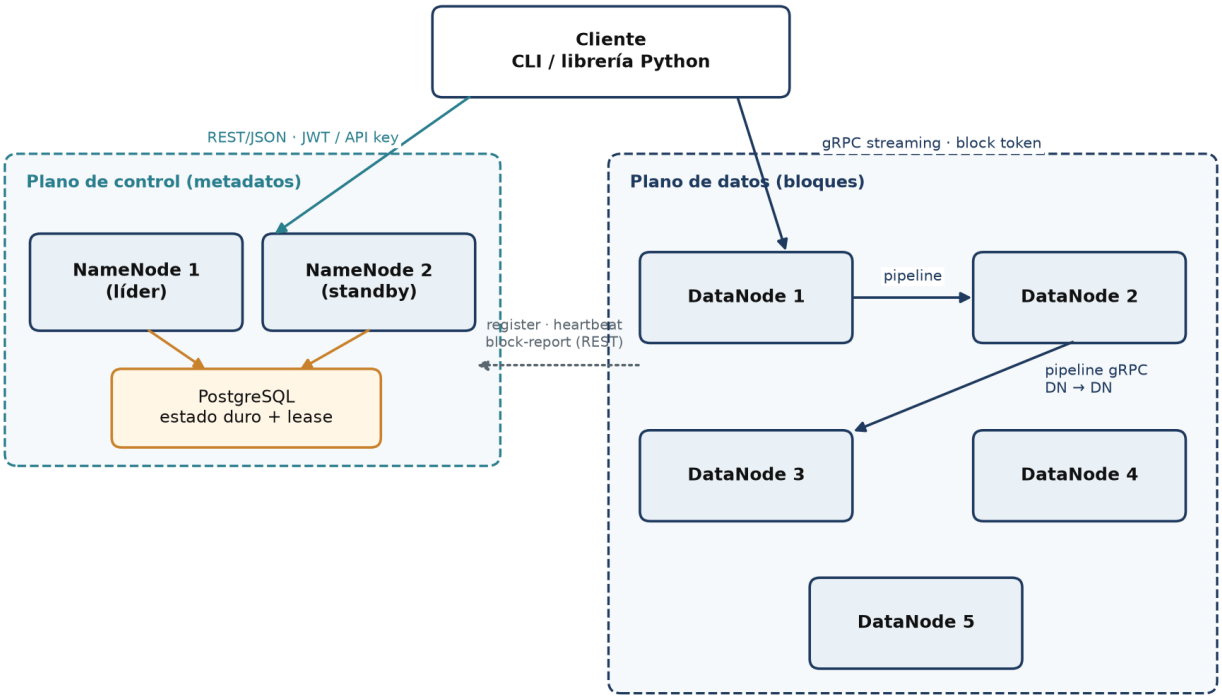


Figura 1. Arquitectura general: plano de control (2 NameNodes + PostgreSQL) y plano de datos (≥ 5 DataNodes).

Por qué REST para metadatos y gRPC para bloques. Los mensajes de control son pequeños y poco frecuentes, así que el costo de REST es irrelevante y se gana depurabilidad (`curl`) y documentación automática (OpenAPI). El canal de bloques mueve los volúmenes grandes: gRPC (HTTP/2 + Protocol Buffers) permite transmitir un bloque en *chunks* y que un DataNode reenvíe al siguiente mientras aún recibe, además de dar un contrato estricto generado desde el archivo `.proto`. El costo es una pila adicional y una depuración más difícil (se usa `grpcurl` con *reflection* en desarrollo).

3.2 Componentes

Componente	Responsabilidad	Código
NameNode	Espacio de nombres, permisos, mapa archivo→bloques→réplicas, colocación, re-replicación, GC, tokens de bloque, autenticación	`server/nodes/namenode/`
DataNode	Almacenar/servir bloques por gRPC, reenviar en pipeline, verificar integridad, registrarse y enviar latidos/informes	`server/nodes/datanode/`
Cliente	CLI + biblioteca: divide en bloques, transfiere en paralelo, verifica checksums, oculta la ubicación	`client/` (vacío por ahora)
Metadatos	Estado duro. Interfaz `MetadataStore`; hoy implementación en memoria, objetivo PostgreSQL	`server/nodes/namenode/store/`
Común	Configuración por entorno, códigos de error (REST↔gRPC), tokens de bloque HMAC, rutas, reloj simulable	`server/common/`

3.3 Flujos principales

Escritura (`put`). (1) El cliente pide al NameNode subir un archivo; el NameNode valida permisos, reserva la ruta (archivo `PENDING`, invisible), calcula bloques y un *pipeline* de DataNodes por bloque y devuelve identificadores y tokens de escritura. (2) El cliente transmite cada bloque por gRPC al primer DataNode del pipeline; este lo reenvía al siguiente a medida que llega, cada nodo verifica el SHA-256, lo guarda de forma atómica y avisa `block-received` al NameNode. (3) El cliente llama a `commit`; el NameNode comprueba que cada bloque tenga al menos `MIN_REPLICAS_COMMIT` réplicas *confirmadas* y checksums coincidentes, y hace visible el archivo (`COMMITTED`) en una sola operación.

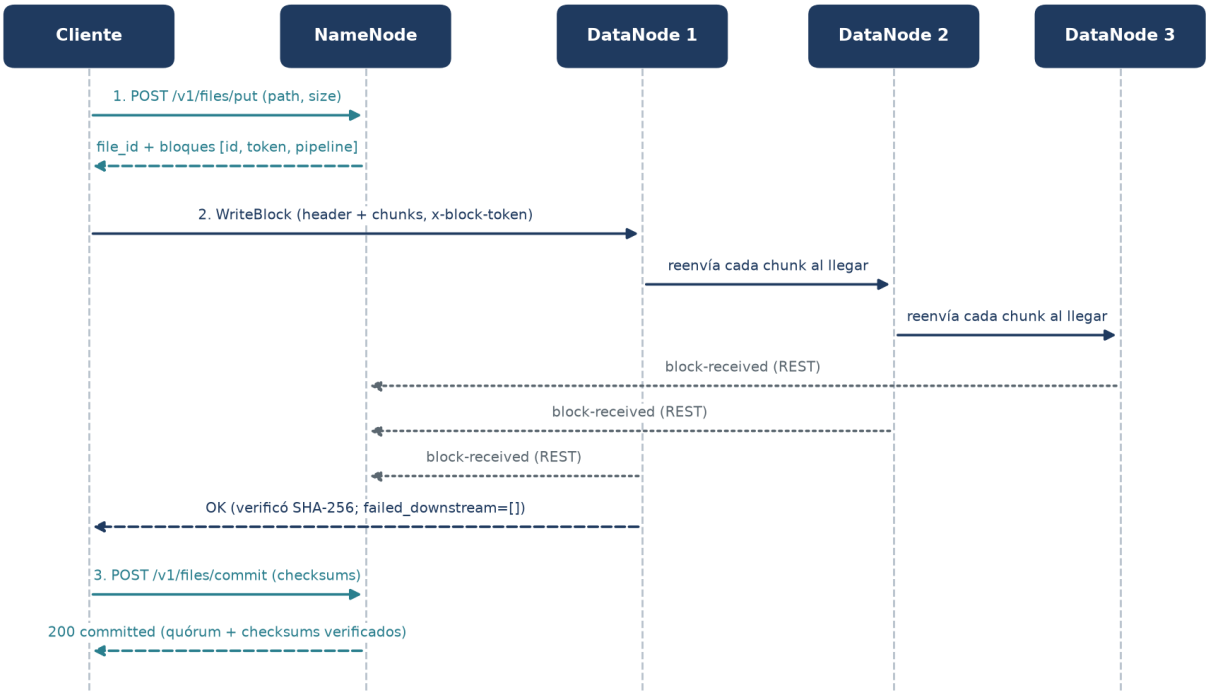


Figura 2. Secuencia de escritura de un bloque a lo largo del pipeline y confirmación (commit).

Lectura (`get`). El NameNode devuelve bloques, checksums, tokens de lectura y las réplicas *vivas* de cada uno. El cliente descarga en paralelo desde cualquier réplica, cae a otra si falla, verifica cada checksum y concatena por índice. Las lecturas parciales usan `offset/length` para no traer el bloque

completo.

Recuperación ante fallos. Un DataNode que deja de enviar latidos por `DEAD_NODE_TIMEOUT_S` (30 s) se marca muerto. Un lazo de fondo busca bloques con menos réplicas vivas que el objetivo y envía órdenes `replicate` en la respuesta al latido de un nodo que tiene copia; el destino confirma con `block-received` hasta restaurar el factor.

3.4 Interior del NameNode

El núcleo (`Namenode`) es una clase de Python puro, sin código web: se puede probar con reloj simulado y almacenamiento en memoria. La capa FastAPI es un adaptador delgado: extrae la identidad, llama al núcleo y traduce `DfshaError` a la respuesta HTTP. Cuatro tareas de fondo mantienen el cluster sano.

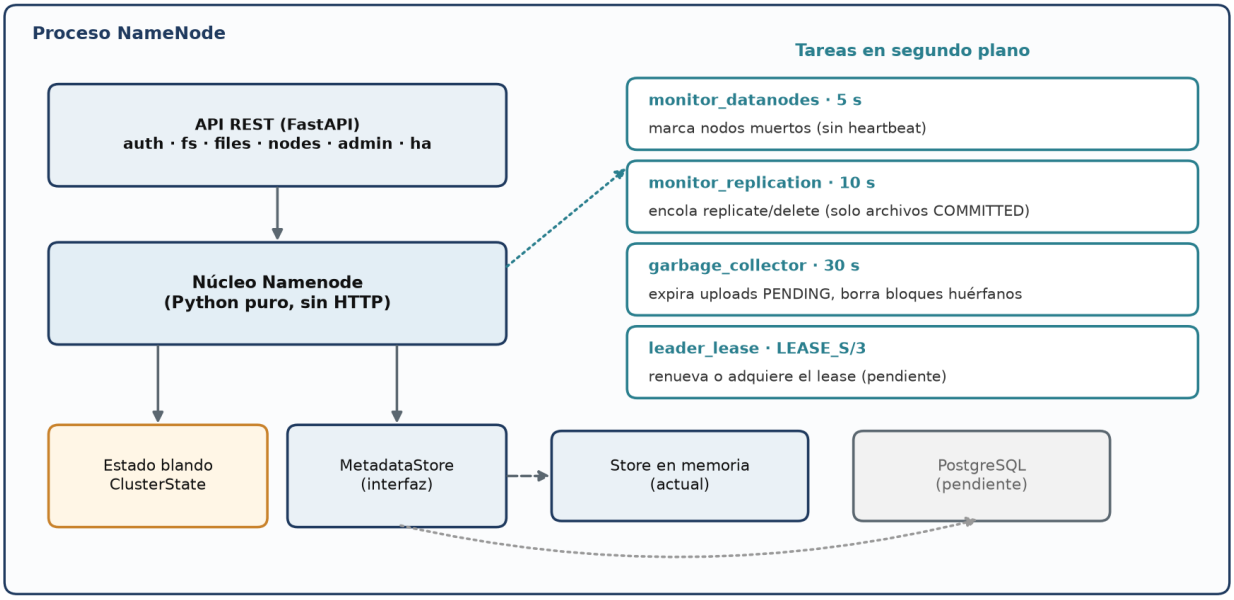


Figura 3. Componentes del proceso NameNode. «Store en memoria» y las 3 primeras tareas están implementadas; PostgreSQL y el lease son trabajo pendiente.

3.5 Modelo de metadatos y estado

La información del NameNode se divide, como en HDFS, en dos clases:

	Estado duro	Estado blando
Contenido	Usuarios, claves de API, grupos, árbol de directorios, archivos y sus bloques (ids, tamaños, checksums, estado), ACL, locks, borrados pendientes, lease del líder, auditoría	Qué DataNodes existen y están vivos, espacio libre, **dónde está cada réplica**, comandos pendientes, replicaciones en curso, pipelines asignados
Dónde vive	Interfaz `MetadataStore` (hoy memoria; objetivo PostgreSQL)	Memoria del líder (`ClusterState`)
¿Se pierde al reiniciar?	No (objetivo)	Sí, y se **reconstruye** con registro, latidos e informes de bloques
Motivo	Debe sobrevivir a cualquier falla y ser visible para ambos NameNodes	Cambia cada pocos segundos y los DataNodes ya guardan la verdad; escribirlo en BD sería sobrecarga

Entidades principales: **FileMetadata** (id UUID4, propietario, tamaño, tamaño de bloque, replicación, estado `PENDING | COMMITTED | ABORTED`, checksum, marcas de tiempo, bloques ordenados), **BlockMetadata** (id opaco, índice, tamaño, checksum SHA-256), **DirectoryMetadata** (hijos y archivos

indexados por nombre) y **DataNodeInfo** (direcciones interna y pública, capacidad, uso, último latido). El nombre completo de una ruta se *calcula* y no se almacena en cada nodo, de modo que `mv` cambia una sola referencia en lugar de reescribir todos los descendientes. La pertenencia de réplicas (`replicas`) solo contiene copias *confirmadas*; el pipeline es únicamente el plan.

Alta disponibilidad del NameNode (diseñada). Dos NameNodes comparten una base PostgreSQL replicada (primario + réplica en caliente; RDS Multi-AZ en la nube). Un único registro `leader(holder, term, lease_expires_at)` actualizado con una sentencia atómica decide el líder; el líder renueva cada LEASE_S/3 y un standby lo reemplaza en ~LEASE_S (15 s). Toda escritura de estado duro verifica el lease en la misma transacción (*fencing* en base de datos). Si la BD cae, el líder **no renuncia**: pasa a modo solo-lectura (lecturas desde su caché, escrituras con 503) porque renunciar dejaría a todo el cluster sin servicio.

Estado actual: **Pendiente** — el repositorio solo expone `GET /v1/ha/status` con líder fijo; la implementación PostgreSQL, el lease y el segundo NameNode corresponden a los issues 03 y 06.

4. Especificación de protocolos y APIs

El contrato oficial entre componentes es `docs/API.md` (v0.9): si una implementación difiere, el contrato gana hasta acordar el cambio. Esta sección resume el contrato; el detalle completo (ejemplos JSON, errores) está en ese documento y en el OpenAPI que genera FastAPI en `/docs`.

4.1 Canales de comunicación

Interacción	Protocolo	Autenticación	Notas
Cliente ↔ NameNode	REST/JSON sobre HTTPS	JWT o API key	Solo metadatos
Cliente ↔ DataNode	gRPC (HTTP/2 + Protobuf) sobre TLS	Token de bloque en metadata `x-block-token`	Bytes del bloque en chunks
DataNode ↔ DataNode	gRPC sobre TLS	Token de bloque; mTLS (objetivo)	Pipeline de escritura y re-replicación
DataNode → NameNode	REST/JSON sobre HTTPS	`X-Node-Key` + mTLS (objetivo)	Registro, latidos, informes; las órdenes viajan en la respuesta
NameNode ↔ NameNode	REST sobre HTTPS	mTLS (objetivo)	`/v1/ha/*`; la coordinación real va por el lease en BD
NameNode ↔ BD	SQL sobre TLS	Credenciales; red privada	Solo el primario se escribe

4.2 Convenciones generales

- **Formato:** JSON UTF-8 en REST; bloques como bytes crudos en chunks por gRPC (nunca JSON ni base64). Fechas ISO-8601 en UTC; tamaños en bytes; unidades binarias (MiB, GiB).
- **Rutas:** absolutas, en **un árbol global**; el hogar de cada usuario es `/users/<usuario>`. La biblioteca cliente muestra el hogar como `/` pero el servidor nunca reescribe rutas.
- **Checksums:** SHA-256 hexadecimal sobre los bytes en claro del bloque, tal como los envía el cliente (antes del cifrado en reposo).
- **Identificadores:** `file_id` UUID4; `block_id` opaco generado por el NameNode (ningún componente debe interpretarlo).
- **Trazabilidad:** `X-Request-Id` (REST) / `x-request-id` (gRPC) se propaga y registra.
- **Compatibilidad:** los receptores ignoran campos desconocidos; los cambios incompatibles requieren `/v2` o decisión del equipo registrada en el changelog.

4.3 API REST del NameNode

Prefijo `/v1`. Rutas implementadas en el repositorio (verificadas en `server/nodes/namenode/api/routes/`):

Grupo	Endpoints	Estado
Autenticación	`POST /auth/register`, `POST /auth/login`, `POST|GET /auth/apikeys`, `DELETE /auth/apikeys/{id}`	**Implementado**
2FA (TOTP)	`/auth/2fa/*`, `/auth/login/2fa`. El login ya devuelve la rama `mfa_required`, pero la verificación TOTP está diferida	**Pendiente**
Archivos	`POST /files/put`, `/files/commit`, `/files/abort`, `GET /files/get`, `POST /files/tokens`	**Implementado**
Sistema de archivos	`GET /fs/ls`, `/fs/stat`, `POST /fs/mkdir`, `/fs/mv`, `DELETE /fs/rm`, `/fs/rmdir`, `GET|PUT /fs/acl`, `POST /fs/lock`, `/fs/lock/{id}/renew`, `DELETE /fs/lock/{id}`	**Implementado**
Grupos	`/groups` (crear grupo, miembros)	**Pendiente**

Grupo	Endpoints	Estado
Nodos (DataNode→NN)	`POST /nodes/register`, `/nodes/heartbeat`, `/nodes/block-report`, `/nodes/block-received`, `/nodes/block-corrupt`	**Implementado**
Administración / HA	`GET /health`, `GET /v1/admin/stats`, `GET /v1/ha/status`. Faltan `/admin/nodes`, `/admin/blocks/{id}` y `/ha/stepdown`	**Parcial**

Ejemplo — subida (`put`). El plan devuelve, por cada bloque, el pipeline ordenado y un token de escritura:

```
POST /v1/files/put
{ "path": "/users/ana/report.pdf", "size": 10485760, "overwrite": false }

201 Created
{ "file_id": "7b0c9e0e-5e0a-4a58-9a11-1f0c6a3d2b11",
  "block_size": 4194304, "replication": 3,
  "blocks": [ { "index": 0, "block_id": "blk_7b0c9e0e_000000", "size": 4194304,
                "token": "<block-token-write>",
                "pipeline": [ {"node_id":"dn1","addr":"dn1:9000","internal_addr":"dn1:9000"},
                              {"node_id":"dn2", ...}, {"node_id":"dn3", ...} ] } ] }
```

Formato de error (único para toda la API REST): `{ "error": { "code": "FILE_NOT_FOUND", "message": "..." } }`. Los códigos principales: 400 `BAD_REQUEST`; 401 `UNAUTHORIZED`; 403 `FORBIDDEN`; 404 `FILE_NOT_FOUND`; 409 `ALREADY_EXISTS`, `UPLOAD_IN_PROGRESS`, `INSUFFICIENT_REPLICAS`, `NODE_NOT_REGISTERED`; 422 `CHECKSUM_MISMATCH`; 423 `LOCKED`; 503 `NOT_ENOUGH_DATANODES`, `NOT_LEADER`. Los mismos nombres se reutilizan en el canal gRPC mediante el metadato final `x-dfsha-error-code`.

4.4 API gRPC de bloques (`BlockService`)

Definida en `proto/block_service.proto` y expuesta por cada DataNode. Los stubs no se versionan: se generan con `python -m grpc_tools.protoc -I. --python_out=. --grpc_python_out=. proto/block_service.proto`.

RPC	Tipo	Descripción	Estado
`WriteBlock`	client-streaming	Primer mensaje: cabecera (`block_id, size, checksum_sha256, pipeline`); siguientes: chunk. Reenvía al siguiente nodo mientras recibe; cada nodo verifica SHA-256 y renombra `.tmp → .dat` de forma atómica. Idempotente (`already_existed`). Devuelve `failed_downstream`	**Implementado**
`ReadBlock`	server-streaming	Lectura por rangos (`offset, length`); cabecera con tamaño y checksum del bloque completo; fuera de rango → `OUT_OF_RANGE`	**Implementado**
`StatBlock`	unario	Tamaño y checksum sin transferir bytes	**Implementado**
`Health`	unario	Estado (`ok|degraded`), espacio libre, número de bloques; no requiere token	**Implementado**

```
service BlockService {
  rpc WriteBlock(stream WriteBlockRequest) returns (WriteBlockResponse);
  rpc ReadBlock(ReadBlockRequest)          returns (stream ReadBlockResponse);
  rpc StatBlock(StatBlockRequest)          returns (StatBlockResponse);
  rpc Health(HealthRequest)                returns (HealthResponse);
}
message WriteBlockHeader { string block_id = 1; int64 size = 2;
                           string checksum_sha256 = 3; repeated string pipeline = 4; }
message WriteBlockRequest { oneof payload { WriteBlockHeader header = 1; bytes chunk = 2; } }
```

Tamaño de chunk: hasta `BLOCK_CHUNK_SIZE` (1 MiB por defecto); un DataNode rechaza chunks de más de 2 MiB, de modo que un bloque nunca viaja como un solo mensaje (el límite por defecto de gRPC es 4 MB).

4.5 Seguridad del canal de bloques: token de bloque

Token = `base64url(payload) + "." + base64url(HMAC-SHA256(secreto, payload))` con payload `{ bid, op ("r"|"w"), usr, exp }`. Un *interceptor* gRPC del DataNode valida localmente (sin consultar al NameNode) la firma, la expiración, que `bid` coincida con el bloque solicitado y que `op` corresponda al RPC. Un token de un bloque no sirve para otro. Vigencia por defecto: 900 s. El secreto compartido solo reside en variables de entorno de NameNodes y DataNodes.

4.6 API de control de los DataNodes (DataNode → NameNode)

El NameNode **nunca** inicia conexiones hacia los DataNodes; las órdenes viajan en las respuestas.

Llamada	Frecuencia	Contenido / respuesta
`register`	Al arrancar	node_id, direcciones interna/pública, capacidad y uso → intervalos y `term`
`heartbeat`	Cada 5 s	Espacio libre/usado → `{ term, commands }` (`replicate`, `delete`)
`block-report`	Al registrarse y cada 60 s	Inventario completo de bloques → órdenes de borrado de huérfanos
`block-received`	Al almacenar un bloque	Confirma la réplica (base del quórum de `commit`)
`block-corrupt`	Al detectar corrupción	El NameNode descarta esa réplica y agenda una nueva

Fencing por término. Cada respuesta lleva el `term` del líder. El DataNode recuerda el mayor visto e **ignora** órdenes de un término menor (líder zombi); al ver un término **mayor**, envía de inmediato un informe completo de bloques para que el nuevo líder reconstruya el mapa de réplicas. Un latido de un nodo desconocido responde `409 NODE_NOT_REGISTERED`: el DataNode se registra de nuevo y reenvía su informe (caso normal tras un failover).

4.7 Parámetros configurables

Variable	Defecto	Función
`BLOCK_SIZE`	64 MiB (demo: 1 MiB)	Tamaño de bloque
`REPLICATION_FACTOR` / `MIN_REPLICAS_COMMIT`	3 / 2	Réplicas objetivo / quórum de escritura para `commit`
`HEARTBEAT_INTERVAL_S` / `DEAD_NODE_TIMEOUT_S`	5 / 30	Latido / tiempo sin latido para declarar muerto un nodo
`BLOCK_REPORT_INTERVAL_S`	60	Periodo del informe de bloques
`TOKEN_TTL_S` / `JWT_TTL_S`	900 / 3600	Vigencia de tokens de bloque / de acceso
`UPLOAD_TTL_S`	3600	Una subida sin `commit` se recolecta pasado este tiempo
`LEASE_S` / `FAILOVER_GRACE_S`	15 / 30	Vigencia del lease / gracia tras un cambio de líder

5. Algoritmos de particionamiento y distribución

5.1 División de un archivo en bloques

`num_bloques = max(1, ceil(tamaño / BLOCK_SIZE))`. Todos los bloques miden `BLOCK_SIZE` excepto el último, que contiene el resto; un archivo vacío genera un bloque de 0 bytes. El uso de `ceil` evita perder el residuo (10 MiB con bloques de 4 MiB son 3 bloques, no 2).

5.2 Colocación por bloque (*placement*)

Implementada en `server/nodes/namenode/placement.py` como funciones puras con la fuente aleatoria inyectada (reproducible con semilla). Para **cada bloque por separado**:

- **Candidatos:** nodos vivos cuyo espacio utilizable (`libre − reservado`) cabe el bloque. Si hay menos que `REPLICATION_FACTOR` → `NOT_ENOUGH_DATANODES`.
- **Muestreo ponderado sin reemplazo** de `REPLICATION_FACTOR` nodos distintos, con probabilidad proporcional al espacio utilizable (algoritmo de Efraimidis–Spirakis: a cada nodo se le asigna la clave $u^{1/w}$ con u uniforme y w el peso, y se conservan las mayores; se calcula como `log(u)/w` para evitar que con pesos de cientos de GB la potencia se redondee a 1.0 y todos empaten).
- **Orden del pipeline:** el nodo con más espacio utilizable va primero (es el que recibe del cliente y carga más).
- **Reserva:** se suma el tamaño del bloque a `reserved_bytes` de cada nodo elegido, para que los bloques siguientes vean el espacio ya comprometido.

El espacio libre favorece a los nodos más vacíos y el azar hace que bloques consecutivos de un mismo archivo caigan en **conjuntos distintos de nodos**: esto es lo que *particiona* el archivo a través del cluster en lugar de duplicarlo entero en todos.

5.3 ¿Por qué se necesitan más DataNodes que el factor de replicación?

El enunciado exige dos propiedades distintas: que los archivos se repartan entre nodos y que *cada archivo* se reparta entre varios nodos. Con exactamente `REPLICATION_FACTOR` nodos el algoritmo no tiene libertad: cada bloque debe ir a todos los nodos y cada DataNode termina con una copia completa de cada archivo, de modo que particionar es indistinguible de espejar. Además la **re-replicación se vuelve imposible**: reparar un bloque exige un nodo vivo que no lo tenga, y con 3 nodos y factor 3 no puede existir. Por eso el despliegue mínimo es de **5 DataNodes**. La Figura 4 lo demuestra ejecutando el código real de placement (sección 7.3).

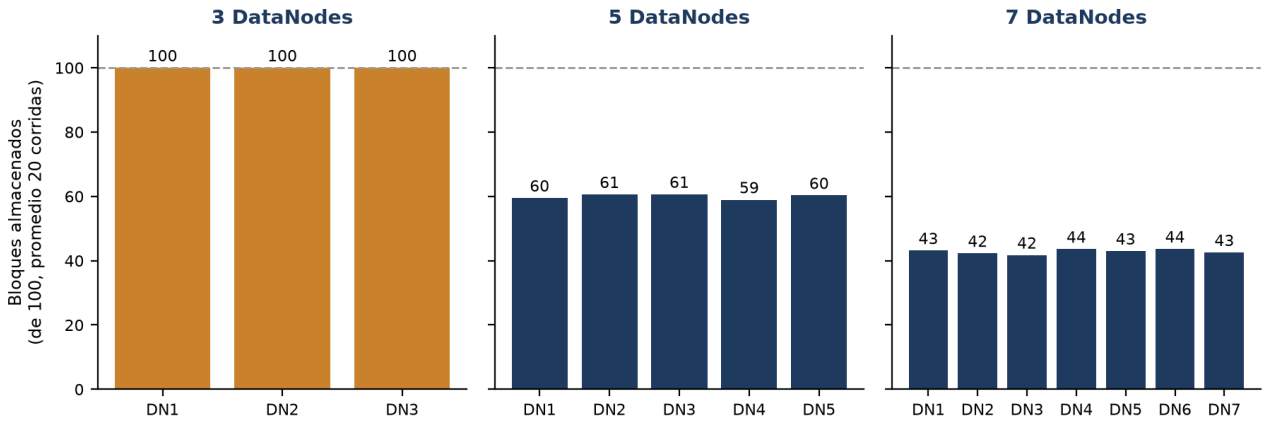


Figura 4. Bloques por DataNode para un archivo de 100 bloques (RF = 3), promedio de 20 corridas con el placement real del repositorio. Con 3 nodos cada uno guarda 100/100 bloques (copia completa); con 5, ≈ 60 %; con 7, ≈ 43 %.

5.4 Asignación versus confirmación

El *pipeline* es solo el plan. Un nodo cuenta como réplica de un bloque cuando confirma (`block-received`) o lo reporta en un informe de bloques. El quórum de escritura (`MIN_REPLICAS_COMMIT`) se comprueba siempre contra réplicas **confirmadas**, nunca contra el pipeline. El código lo refleja con dos estructuras separadas: `pipelines` (plan) y `replicas` (confirmaciones).

5.5 Tolerancia a un nodo caído en el pipeline

Un pipeline es una cadena: un nodo caído en el *medio* cortaría a todos los de atrás. Medido con una cadena de 3 nodos y el del medio apagado, sin tratamiento se obtenía 1 réplica en lugar de 3 (menos que el quórum), obligando a resubir el bloque. Por eso un DataNode que no alcanza a `pipeline[0]` **prueba los siguientes en orden** y reenvía al primero que responda, pasándole el resto de la cadena; si el reenvío falla a mitad de la transferencia, reproduce la copia local desde disco (sin cargar el bloque entero en memoria). Las direcciones omitidas se devuelven en `failed_downstream`. Tres réplicas degradan a dos, el quórum se cumple y la re-replicación restaura la tercera. **Estado: Implementado** y cubierto por pruebas de integración (sección 7.2).

5.6 Re-replicación

- Solo se examinan bloques de archivos `COMMITTED` (nunca de subidas en curso).
- Prioridad: primero los bloques con menos réplicas vivas.
- Origen: nodo vivo que tiene el bloque. Destino: nodo vivo con espacio que *no* lo tiene.
- La orden viaja en el latido del nodo **origen**; no se vuelve a encolar el mismo par (bloque, destino) hasta que expire el intento.
- Réplicas sobrantes (por ejemplo, un nodo que vuelve) reciben una orden `delete`.

5.7 Asignación incremental de bloques (diseñada)

Asignar todos los bloques en el `put` hace crecer la respuesta con el tamaño del archivo (≈ 16 000 bloques para 1 TB con bloques de 64 MiB), caduca los tokens de los últimos bloques durante subidas largas, decide la colocación con información obsoleta y reserva capacidad del cluster durante toda la subida. El diseño propone **asignar por lotes** a medida que avanza la subida (8–16 bloques por lote, con tokens y pipelines frescos), declarando el tamaño total en el `commit`. **Estado: Diseñado**; hoy el `put` asigna todo por adelantado y existe `MAX_FILE_SIZE_BYTES` (10 GiB) como límite.

6. Entorno de ejecución (nativo y Docker)

6.1 Ejecución nativa (disponible hoy)

Cada nodo se ejecuta como proceso Python desde la raíz del repositorio. Se probó con Python 3.12 y las dependencias de `requirements.txt` (versiones exactas): FastAPI, Uvicorn, Pydantic, argon2-cffi, PyJWT, pyotp, qrcode, httpx, grpcio, grpcio-tools, protobuf y psycopg (reservado para el objetivo PostgreSQL).

```
python -m venv .venv && source .venv/bin/activate        # (Windows: .venv\Scripts\Activate.ps1)
pip install -r requirements.txt
python -m grpc_tools.protoc -I. --python_out=. --grpc_python_out=. proto/block_service.proto

# Plano de control: el NameNode exige BLOCK_TOKEN_SECRET (no arranca sin él)
export BLOCK_TOKEN_SECRET=...  JWT_SECRET=...  NODE_KEY=...
python -m server.nodes.namenode

# Plano de datos: un DataNode por terminal (NAMENODE_URLS por defecto: http://localhost:8000)
python -m server.nodes.datanode --id dn1 --port 9001 --storage ./data/dn1
python -m server.nodes.datanode --id dn2 --port 9002 --storage ./data/dn2 --namenode-urls http://localhost:8000

# Pruebas
python -m pytest -q
```

La configuración se hace **solo con variables de entorno** (sin secretos en el repositorio): secretos `BLOCK_TOKEN_SECRET`, `JWT_SECRET` y `NODE_KEY` (cabecera `X-Node-Key`), usuario inicial `ADMIN_USER`/`ADMIN_PASSWORD`, además de los parámetros de la sección 4.7. El módulo `server/common/config.py` valida los valores y falla de forma explícita ante datos inválidos.

6.2 Docker y topología de despliegue (objetivo)

Cada nodo puede correr nativo o en contenedor. El perfil local previsto es un `docker-compose.yml` con `postgres-primary`, `postgres-replica`, `namenode1`, `namenode2`, `datanode1..5` y un contenedor `client`, con perfil de demostración de bloques de 1 MiB. Para la nube (AWS Academy / GCP) se propone la siguiente topología:

Máquina	Ejecuta	Puertos
vm1	NameNode 1	8000 (REST: clientes y DataNodes)
vm2	NameNode 2	8000 (REST)
vm3 … vm7	DataNode 1 … 5	9000 (gRPC: clientes y otros DataNodes)
administrada	RDS Multi-AZ PostgreSQL	5432 (solo desde el grupo de seguridad de los NameNodes)

Los grupos de seguridad restringen la base de datos a los NameNodes y exponen únicamente el puerto REST del NameNode y el puerto gRPC de los DataNodes. Los stubs gRPC se generan en la construcción de la imagen (no se versionan).

> Estado: **Pendiente** — el repositorio aún no contiene `docker-compose.yml`, Dockerfiles ni `.env.example` (issue 09). Este apartado se completará con la configuración real y los comandos de despliegue cuando se integre.

7. Pruebas y análisis de resultados

Todas las cifras de esta sección se obtuvieron ejecutando el código del commit `0bcb216` en un entorno limpio (Python 3.12, dependencias de `requirements.txt`).

7.1 Suite automática

Resultado: **412 pruebas, 412 aprobadas, 0 fallos** (≈ 19 s). Las pruebas del núcleo usan reloj simulado, almacenamiento en memoria y aleatoriedad con semilla, por lo que no necesitan red ni base de datos.

Módulo	Pruebas	Qué verifica
Núcleo NameNode — subida	73	Reserva de ruta, `UPLOAD_IN_PROGRESS`, `overwrite`, quórum y checksums en `commit`, idempotencia
Núcleo NameNode — almacén de metadatos	39	Contrato de `MetadataStore`: árbol, índices, estados
Núcleo NameNode — registro / replicación / informes	30 / 22 / 16	Registro y latidos de nodos, re-replicación, informes de bloques
Núcleo NameNode — lectura y FS / placement / GC / stats	29 / 21 / 12 / 9	Lectura, `mv`, locks; ponderación; recolección; estadísticas y lazos
Autenticación y ACL	23	Contraseñas, JWT, API keys, ACL por usuario/grupo
API REST	5	Casos de extremo a extremo con `TestClient`
Común (config, errores, rutas, tokens, reloj)	96	Validación de configuración, mapeo de errores REST↔gRPC, tokens HMAC, normalización de rutas
Integración del plano de datos (gRPC)	30	Pipeline multi-nodo en proceso, checksum erróneo, nodo caído en cada posición, lecturas por rangos, `StatBlock`, interceptor de tokens, idempotencia y concurrencia, chunk sobredimensionado, corrupción
Bucle de control y almacenamiento del DataNode	7	Registro, latidos, informes, almacenamiento en disco

Cobertura de líneas medida con `pytest-cov` solo sobre el código de producción (excluyendo los archivos de prueba): **84,9 %** (2 410 de 2 838 sentencias). Núcleo del NameNode 87 %, `block_service.py` 82 %, `placement.py` 100 %, `common/` 96–100 %. Zonas con menor cobertura: `datanode/core.py` (63 %, lazos de control), `routes/fs.py` (53 %) y `datanode/__main__.py` (0 %, punto de entrada).

7.2 Escenarios de fallo en el plano de datos (pruebas de integración)

Escenario	Resultado verificado
Bloque replicado a lo largo del pipeline	El bloque aparece idéntico en todos los nodos de la cadena
Checksum incorrecto	Rechazado por `INVALID_ARGUMENT`; no queda ningún archivo (ni temporal)
Nodo posterior caído	Se reporta en `failed_downstream`; la copia local sobrevive
Nodo del **medio** caído	Se omite y el bloque llega al siguiente nodo vivo
Primer o último salto caído; todos no disponibles	Se reportan y la copia local se conserva
Lectura por rangos y rangos inválidos	Se sirven exactamente los bytes pedidos con el checksum del bloque completo; inválidos → `OUT_OF_RANGE`
Token ausente, inválido, expirado o de otro bloque	Rechazado por el interceptor (`UNAUTHENTICATED` / `PERMISSION_DENIED`)
Escritura duplicada y concurrente	Idempotente; un solo bloque válido; contenido distinto con mismo id es rechazado
Bloque corrupto en disco al leerlo completo	`DATA_LOSS` con metadatos de error

7.3 Evidencia de particionamiento (simulación con el placement real)

Se ejecutó 20 veces (semillas 0–19) `choose_pipeline` del repositorio para un archivo de 100 bloques de 1 MiB (perfil demo), factor de replicación 3, nodos de 100 GiB. La tabla muestra el porcentaje promedio de los bloques del archivo que almacena cada nodo.

DataNodes	Bloques por nodo (promedio)	Fracción del archivo por nodo	¿Algún nodo tiene el archivo completo?	¿Re-replicación posible?
3	100 / 100 / 100	100 %	**Sí: todos** (espejo)	No (no hay nodo libre de la réplica)
5	59,5 – 60,6	≈ 60 % (3/5)	No	Sí
7	41,8 – 43,6	≈ 43 % (3/7)	No	Sí

Análisis. El reparto sigue la expectativa teórica RF/N (3/5 = 60 %, 3/7 ≈ 43 %), con diferencias entre nodos de uno o dos puntos porcentuales, lo que confirma que el muestreo ponderado equilibra la carga. Con 3 nodos el sistema funciona pero *refleja* en lugar de particionar, y no puede reparar réplicas. Esto justifica el mínimo de 5 DataNodes y es la evidencia que se mostrará en el escenario T2 del plan de pruebas.

7.4 Plan de pruebas de sistema (pendiente de ejecutar sobre cluster)

El plan completo (T1–T24) está en `docs/ARCHITECTURE.md` §12 y requiere el cluster integrado. Resumen de los escenarios y su estado:

Escenario	Qué se comprobará	Estado
T1 · put/get de 100 MB y comparación SHA-256	Integridad de extremo a extremo	**Pendiente**
T2 · distribución de bloques en cluster de 5 nodos	Partición y replicación como propiedades separadas	**Parcial**
T3/T4 · caída de un DataNode (lectura y re-replicación)	Lectura sin error; `under_replicated_blocks` sube y vuelve a 0	**Parcial**
T5/T16/T17 · caída del NameNode líder, a mitad de `put`, reinicio total	Failover en ~`LEASE_S`; datos intactos	**Pendiente**

Escenario	Qué se comprobará	Estado
T6 · corrupción de bloque en disco	Detección, lectura desde otra réplica y reparación	**Parcial**
T7/T8/T14/T18 · concurrencia, overwrite, locks, subida abandonada	Un solo escritor, visibilidad atómica, 423 `LOCKED`, GC	**Parcial**
T9–T12 · ACL, tokens, cifrado en reposo, TLS	Controles de seguridad	**Parcial**
T13 · agregar un DataNode	Nuevos bloques llegan al nodo nuevo	**Pendiente**
T15 · experimentos de caudal (1, 2, 4, 8 transferencias; 3/5/7 nodos; tamaños de bloque y de chunk)	Tablas y gráficos de rendimiento	**Pendiente**
T20–T24 · caída de PostgreSQL, 2FA, mTLS, nodo del medio del pipeline, lecturas por rango	Modo solo-lectura, seguridad, resiliencia	**Parcial**

«Parcial» indica que la lógica correspondiente ya está cubierta por pruebas unitarias o de integración en proceso (secciones 7.1–7.2) pero el escenario aún no se ha corrido sobre el cluster real.

7.5 Mediciones de desempeño (por completar)

Los experimentos de la sección 11 de `ARCHITECTURE.md` (caudal con 1/2/4/8 transferencias paralelas, 3 vs 5 vs 7 nodos, bloques de 1/4/64 MiB, chunks de 256 KiB/1/2 MiB, factor 1 vs 3, operaciones de metadatos bajo carga, tiempos de re-replicación y de failover) requieren el cliente y el cluster integrados. Aquí se incluirán las tablas y gráficos con los datos crudos cuando se ejecuten.

8. Estado de implementación y trabajo pendiente

8.1 Resumen por capacidad

Capacidad	Estado	Detalle
Núcleo NameNode (subida, commit, lectura, FS, ACL, locks, GC, re-replicación)	**Implementado**	Con pruebas; metadatos en memoria
API REST del NameNode	**Implementado**	Auth, archivos, FS, nodos, admin/stats, ha/status
Placement ponderado por bloque	**Implementado**	100 % de cobertura; evidencia en 7.3
DataNode: gRPC (Write/Read/Stat/Health), pipeline, rangos, tokens	**Implementado**	Almacenamiento atómico con checksum sidecar
DataNode: registro, latidos, informes, fencing por término, outbox	**Implementado**	Reintentos ante cortes con el NameNode
Autenticación: Argon2id, JWT, API keys	**Implementado**	—
Metadatos en PostgreSQL + réplica	**Pendiente**	Issue 03
Lease, `term` y segundo NameNode (HA)	**Pendiente**	Issue 06
TLS / mTLS	**Pendiente**	Hoy los canales son inseguros (Issue 07)
Cifrado en reposo AES-256-GCM	**Pendiente**	Hoy los bloques se guardan en claro (Issue 07)
2FA (TOTP)	**Pendiente**	Rama `mfa_required` presente; verificación diferida
Grupos de usuarios y endpoints de admin restantes	**Pendiente**	—
Cliente (biblioteca y CLI)	**Pendiente**	`client/` solo contiene `__init__.py` (Issue 08)
Asignación incremental de bloques	**Diseñado**	Requiere cambio de contrato (versión nueva)
Docker Compose y despliegue en la nube	**Pendiente**	Issue 09
Ejecución del plan de pruebas y mediciones	**Pendiente**	Issue 10
Métrica `active_transfers` y *scrubber* de disco	**Pendiente**	Mejoras del DataNode

8.2 Diferencias detectadas entre el contrato y el código (a resolver)

Al revisar el repositorio encontramos puntos donde el código y los documentos no coinciden. Conviene decidirlos pronto para que el cliente se construya contra una sola versión:

Tema	Contrato (`API.md`)	Código actual
Ruta de `commit/abort/tokens`	`POST /v1/files/{file_id}/commit` (y análogos)	`POST /v1/files/commit` con `file_id` en el cuerpo
Renovación de tokens	Cuerpo `{ indexes: [...] }`; devuelve bloques con pipelines frescos	Cuerpo `{ block_ids: [...] }`; devuelve solo un mapa de tokens de escritura
Cuerpo de `commit`	Incluye `file_checksum` opcional	Solo `file_id` y bloques (sin checksum de archivo)
Respuesta de `get`	Incluye `block_size` y `checksum` del archivo	No los incluye
Respuesta de `abort`	`200 {}`	`204` sin cuerpo
Formato de error	`{ error: { code, message } }`	Igual. Nota: `docs/REST_API.md` menciona RFC 7807, que no es lo implementado

Otras observaciones: el contrato vigente es la v0.9, pero la cabecera de `docs/ARCHITECTURE.md` hace referencia a la v0.8; y el documento de arquitectura describe la asignación incremental (§6.3) como

diseñada mientras que `API.md` todavía documenta el `put` con asignación completa.

8.3 Limitaciones declaradas del diseño

- El NameNode líder es el único escritor de metadatos (cuello de botella a muy gran escala; trabajo futuro: federación por prefijo de ruta, réplicas de lectura).
- Si se pierden ambos nodos de la base de datos, el servicio queda en solo-lectura sin poder aceptar cambios.
- Tras un failover hay una ventana corta (`FAILOVER_GRACE_S` + informes de bloques) en la que `commit` puede requerir reintento.
- Solo WORM: no hay actualizaciones parciales in situ; modificar equivale a subir una nueva versión.
- 2FA protege el inicio de sesión interactivo; las claves de API quedan exentas por diseño.
- No hay montaje POSIX (FUSE): la transparencia de acceso es dinámica, mediante CLI y biblioteca.
- El canal de bloques usa gRPC: no se puede invocar desde un navegador ni con `curl`; cualquier balanceador intermedio debe soportar HTTP/2 de extremo a extremo.

8.4 Próximos pasos para completar este informe

- Integrar PostgreSQL, el lease y el segundo NameNode; actualizar las secciones 3.5 y 4.
- Integrar TLS/mTLS y AES-256-GCM; actualizar la sección 4 (seguridad) y el cierre del análisis de amenazas.
- Completar el cliente y la CLI; ejecutar T1–T24 y los experimentos de rendimiento; llenar las secciones 7.4 y 7.5 con datos reales.
- Añadir `docker-compose.yml` y la topología desplegada en la nube; completar la sección 6.
- Resolver las diferencias de la sección 8.2 y regenerar el documento.

Referencias

- Ghemawat, S., Gobioff, H. y Leung, S.-T. *The Google File System.* ACM SOSP, 2003.
- Shvachko, K., Kuang, H., Radia, S. y Chansler, R. *The Hadoop Distributed File System.* IEEE MSST, 2010.
- M'Raihi, D. et al. *TOTP: Time-Based One-Time Password Algorithm.* RFC 6238, IETF, 2011.
- Efraimidis, P. S. y Spirakis, P. G. *Weighted random sampling with a reservoir.* Information Processing Letters, 97(5), 2006.
- Documentación del proyecto: `docs/API.md`, `docs/ARCHITECTURE.md`, `docs/NAMENODE_MODELS.md`, `docs/GRPC_BLOCK_SERVICE.md`, `docs/DATANODE.md`, `docs/REST_API.md`.
- gRPC Authors. *gRPC Documentation* y Protocol Buffers Language Guide (proto3).